

Network Protocols & Wireshark

1. Install and configure a basic Apache or Nginx web server on your local machine, then verify it is running by accessing `http://localhost` in your browser

->**Install and Configure Apache Web Server on Linux**

1. Install Apache

On Ubuntu/Debian, open Terminal and run:

```
sudo apt update
```

```
sudo apt install apache2 -y
```

2. Start the Apache service

```
sudo systemctl start apache2
```

Enable Apache to start automatically when the computer boots:

```
sudo systemctl enable apache2
```

Check its status:

```
sudo systemctl status apache2
```

You should see:

```
Active: active (running)
```

3. Create a Simple Web Page

Edit the default Apache page:

```
sudo nano /var/www/html/index.html
```

Enter:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My Local Web Server</title>
```

```
</head>

<body>

    <h1>Apache Web Server is Running</h1>

    <p>This page is hosted locally on my computer.</p>

</body>

</html>
```

Save the file.

4. Verify in the Browser

Open your web browser and enter:

http://localhost

You should see:

Apache Web Server is Running

and:

This page is hosted locally on my computer.

You can also test from Terminal:

curl http://localhost

5. Expected Result

Item	Result
Web Server	Apache
Protocol	HTTP
Address	http://localhost
Port	80
Document Root	/var/www/html
Status	Running

Conclusion

Apache was successfully installed and configured as a local web server. Accessing <http://localhost> in the browser displays the custom HTML page, confirming that the web server is running correctly.

2. Use Wireshark to capture HTTP traffic while loading your web server homepage, then identify and list the source and destination IP addresses and the HTTP request method used.

->Wireshark HTTP Traffic Capture

You can capture the traffic generated when opening your local Apache homepage at <http://localhost>.

1. Start Wireshark

1. Open **Wireshark**.
2. Select the **Loopback** interface (lo on Linux or Loopback on Windows).
3. Click **Start Capturing**.
4. Open your browser and visit:

<http://localhost>

5. Return to Wireshark and stop the capture.

2. Filter HTTP Traffic

In the Wireshark display filter, enter:

http

If your system shows the traffic as TCP rather than HTTP, try:

tcp.port == 80

3. Identify the HTTP Request

Select the HTTP packet containing:

GET / HTTP/1.1

The GET is the **HTTP request method** used by the browser to request the homepage.

For a local web server, the packet will typically look like:

Field	Example
Source IP	127.0.0.1
Destination IP	127.0.0.1
HTTP Method	GET
Requested Resource	/
HTTP Version	HTTP/1.1
Destination Port	80

4. What to Write in Your Assignment

Observation:

Wireshark captured HTTP traffic between the browser and the local Apache web server. Since the server was accessed using localhost, both the source and destination addresses were **127.0.0.1** (IPv4 loopback). The browser used the **GET** HTTP request method to request the homepage /.

Result:

Source IP : 127.0.0.1

Destination IP : 127.0.0.1

HTTP Method : GET

Requested Page : /

Port : 80

If your Wireshark capture shows ::1 instead, the browser is using IPv6 loopback rather than IPv4. In that case, record ::1 as both source and destination.

3. Transfer a small text file from your computer to a remote system using FileZilla (FTP client) with both FTP and SFTP, and capture the network traffic for each protocol in Wireshark. Compare and describe the difference in packet details between FTP and SFTP.

Hint: Look for visible file content or credentials in FTP packets versus encrypted data in SFTP packets.

->FTP vs SFTP File Transfer with Wireshark

For this practical, use a **test/VM server that you are authorized to access**. Create a small file such as test.txt containing:

This is a test file for FTP and SFTP transfer.

1. Transfer Using FTP

In **FileZilla**, connect to your FTP test server and upload test.txt.

In Wireshark, capture the traffic and use:

ftp

or:

tcp.port == 21

FTP normally uses **TCP port 21** for the control connection.

You may see readable protocol information such as:

USER username

PASS *****

STOR test.txt

Depending on the FTP setup, usernames, commands, and potentially file contents can be visible because ordinary FTP does **not encrypt the connection**.

2. Transfer Using SFTP

Connect to the same type of test environment using **SFTP** in FileZilla and upload the same test.txt.

In Wireshark, filter:

tcp.port == 22

SFTP operates through **SSH**, normally using **TCP port 22**.

Instead of readable FTP commands or file contents, Wireshark will generally show encrypted SSH packets, for example:

SSH Protocol

Encrypted packet

The actual username, password, commands, and transferred file contents are not visible as plaintext in the packet capture.

3. Comparison

Feature	FTP	SFTP
Full name	File Transfer Protocol	SSH File Transfer Protocol
Common port	TCP 21	TCP 22
Encryption	No encryption by default	Encrypted through SSH
Credentials	Can be visible in plaintext	Encrypted
File contents	Can potentially be observed	Encrypted
Wireshark visibility	FTP commands may be readable	Mostly encrypted SSH data
Security	Less secure	More secure

4. Assignment Result

FTP observation: Wireshark can identify the FTP control connection and display commands such as USER, PASS, and STOR in readable form when ordinary unencrypted FTP is used.

SFTP observation: Wireshark identifies an SSH connection on port 22, but the authentication information, commands, and file contents appear as encrypted data rather than readable text.

Conclusion: The major difference is **encryption**. Standard FTP sends its control information without encryption, while SFTP protects the transfer using SSH encryption. Therefore, SFTP provides confidentiality for credentials and transferred data.

4. Analyze a public website (such as <https://www.flipkart.com>) in Wireshark and find a packet that uses HTTPS. Take a screenshot of the packet details and explain why you cannot view the actual data being sent

->Wireshark Analysis of HTTPS Traffic

You can use a public website such as Flipkart for this practical.

1. Start Wireshark

1. Open **Wireshark**.

2. Select the network interface currently being used.
3. Start the capture.
4. Open the website in your browser.
5. Stop the capture after the page loads.

2. Find an HTTPS Packet

In the Wireshark filter box, enter:

tls

You can also use:

tcp.port == 443

Look for packets showing something similar to:

Transport Layer Security

TLSv1.3

Application Data

A typical packet may show:

Field	Example
Source IP	Your computer's IP
Destination IP	Website/CDN server IP
Protocol	TLSv1.3
Destination Port	443
Information	Application Data

3. Screenshot

For your assignment, select a packet labeled **Application Data** and expand:

Transport Layer Security → Application Data

Take a screenshot showing the packet list and expanded packet details.

Your screenshot should look conceptually like:

No.	Source	Destination	Protocol	Info

125	192.168.1.10	xxx.xxx.xxx.xxx	TLSv1.3	Application Data
126	xxx.xxx.xxx	192.168.1.10	TLSv1.3	Application Data

Use the **actual IP addresses displayed in your Wireshark capture** rather than the example addresses above.

4. Why Can't You See the Actual Data?

HTTPS uses **TLS (Transport Layer Security)** to encrypt the communication between your browser and the website.

After the TLS connection is established:

- HTTP requests and responses are encrypted.
- Login information and other sensitive information are protected.
- The webpage content being transferred cannot normally be read directly from the packet capture.
- Wireshark can show packet metadata such as IP addresses, ports, packet sizes, and TLS information, but the application data appears encrypted.

Assignment Answer

Observation: Wireshark captured traffic to the public website using **TLS/HTTPS over TCP port 443**. The packet was identified as **TLS Application Data**.

Explanation: The actual HTTP data cannot be viewed because HTTPS uses TLS encryption. The browser and web server establish encryption keys during the TLS handshake, and subsequent application data is encrypted. Therefore, a normal Wireshark capture shows encrypted TLS data rather than the readable webpage request or response.

Result:

Protocol: HTTPS / TLS

Port: 443

Packet: TLS Application Data

Actual HTTP data: Encrypted

Reason: TLS encryption

5. Write a short protocol analysis report (200-300 words) summarizing what you observed about HTTP, HTTPS, FTP, and SFTP traffic in Wireshark, focusing on how encryption changes what you can see in the packet captures.

->Here is a 200–300 word report you can submit:

Protocol Analysis Report: HTTP, HTTPS, FTP, and SFTP

During the Wireshark analysis, I observed differences in how HTTP, HTTPS, FTP, and SFTP traffic appears in packet captures. The main difference was whether the communication was encrypted.

HTTP traffic was captured while accessing a local web server using `http://localhost`. Wireshark could identify the HTTP protocol and display information such as the GET request, requested resource, source and destination IP addresses, and port 80. Because basic HTTP does not encrypt application data, the contents of requests and responses can potentially be viewed directly in a packet capture.

HTTPS traffic was observed while accessing a public website. The communication used TLS over port 443. Wireshark could show information such as IP addresses, ports, TLS versions, and encrypted application-data packets. However, the actual HTTP requests, responses, and webpage content were not readable because TLS encrypts the application data.

FTP traffic was also examined during a file transfer. Standard FTP commonly uses port 21 for its control connection and does not encrypt the control channel by default. Therefore, commands and other information may be visible as readable text in Wireshark.

SFTP was different because it operates through SSH, normally on port 22. Authentication information, commands, and transferred file data are encrypted, so Wireshark mainly showed encrypted SSH/TCP traffic rather than readable file contents.

Overall, the captures demonstrated that encryption significantly reduces the amount of application-level information visible to someone analyzing network traffic. Unencrypted protocols expose more readable information, while HTTPS and SFTP protect the contents of communication through encryption.